

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS — 1ère année — Département de français

Durée : 1h30

Année universitaire : 2025-2026

Objectifs pédagogiques spécifiques

À l'issue de cette séance, l'étudiant est capable de :

- Définir un algorithme et énumérer ses 5 caractéristiques fondamentales
- Utiliser des variables, des types de données et des expressions dans un pseudo-code
- Écrire des structures conditionnelles (si/sinon/selon)
- Utiliser des boucles conditionnelles (tant que) et comptées (pour)
- Analyser la complexité temporelle d'un algorithme simple ($O(1)$, $O(n)$, $O(n^2)$)
- Tracer manuellement l'exécution d'un algorithme pas à pas

Plan du cours

1. Qu'est-ce qu'un algorithme ? Définition et caractéristiques (10 min)
2. Variables, types et expressions (15 min)
3. Structures conditionnelles : si, sinon, selon (15 min)
4. Boucles : tant que, pour (20 min)
5. Complexité algorithmique : $O(1)$, $O(n)$, $O(n^2)$ (15 min)
6. Trace d'exécution manuelle (10 min)
7. Applications pédagogiques pour SNT/NSI (5 min)

1. Qu'est-ce qu'un algorithme ?

Définition : Un algorithme est une suite finie d'instructions non ambiguës qui, étant données des entrées, produit des sorties et se termine en un temps fini.

5 caractéristiques fondamentales

1. Finitude : l'algorithme se termine après un nombre fini d'étapes (pas de boucle infinie)
2. Non-ambiguïté : chaque instruction est claire et sans interprétation multiple
3. Entrées : zéro ou plusieurs données d'entrée
4. Sorties : un ou plusieurs résultats produits
5. Efficacité : l'algorithme utilise des ressources (temps, mémoire) raisonnables

Exemple fondateur : algorithme d'Euclide (PGCD)

Origine : Du mathématicien perse Al-Khwârizmî (IXe s.).

```

Algorithme PGCD(a, b)
TANT QUE b ≠ 0 FAIRE
  r ← a MOD b; a ← b; b ← r
FIN TANT QUE
RETOURNER a

graph TD
DEBUT("Début") --> ENTREE["Entrer a, b"]
ENTREE --> COND{"b = 0 ?"}
COND -- OUI --> SORTIE["Afficher a (PGCD)"]
SORTIE --> FIN("Fin")
COND -- NON --> CALC["r ← a MOD b<br/>a ← b<br/>b ← r"]
CALC --> COND

style DEBUT fill:#2ecc71,color:#fff
style ENTREE fill:#3498db,color:#fff
style COND fill:#f39c12,color:#000
style SORTIE fill:#3498db,color:#fff
style FIN fill:#e74c3c,color:#fff
style CALC fill:#9b59b6,color:#fff

```

2. Variables, types et expressions

Une variable est un espace mémoire nommé contenant une valeur qui peut changer.

```

âge ← 25
prénom ← "Alice"
est_majeur ← VRAI

```

Type	Signification	Exemples
Entier (int)	Nombre entier	42, -3, 0
Réel (float)	Nombre à virgule	3.14, -0.5
Booléen (bool)	Vrai ou Faux	VRAI, FAUX
Caractère (char)	Un seul caractère	'A', 'é'
Chaîne (string)	Texte	"Bonjour"

Opérateurs :

- Arithmétiques : +, -, ×, /, MOD (reste), DIV (division entière)
- Comparaison : =, ≠, <, >, ≤, ≥
- Logiques : ET (AND), OU (OR), NON (NOT)

```

moyenne ← (note1 + note2 + note3) / 3
est_pair ← (x MOD 2 = 0)
majeur ← (âge ≥ 18) ET (nationalité = "française")

```

3. Structures conditionnelles

3.1 Si / Sinon (if / else)

```

graph TD
COND2{"Condition ?"} -- VRAI --> B1["Bloc d'instructions 1"]
COND2 -- FAUX --> B2["Bloc d'instructions 2"]
B1 --> SUITE
B2 --> SUITE
style COND2 fill:#f39c12,color:#000

```

```

SI note ≥ 10 ALORS
AFFICHER "Admis"
SINON
AFFICHER "Ajourné"
FIN SI

```

Avec SI/SINON SI :

```

SI note ≥ 16 : "Très bien" ; SINON SI note ≥ 14 : "Bien"
SINON SI note ≥ 12 : "Assez bien" ; SINON SI note ≥ 10 : "Passable" ; SINON : "Ajourné"
FIN SI

```

3.2 Selon / Cas (switch / case)

```

SELON note FAIRE
CAS "A": "Excellent" ; CAS "B": "Bien" ; CAS "C": "Assez bien"
CAS "D": "Passable" ; CAS "E": "Insuffisant" ; DEFAUT: "Invalide"
FIN SELON

```

4. Boucles

4.1 Boucle TANT QUE (while)

Répète un bloc d'instructions tant que la condition est vraie. Le test est effectué avant chaque itération.

```

i ← 1
TANT QUE i ≤ 10 FAIRE
AFFICHER i
i ← i + 1
FIN TANT QUE

```

Exemple — somme des N premiers entiers :

```

Algorithme Somme(N)
somme ← 0; i ← 1
TANT QUE i ≤ N FAIRE
somme ← somme + i; i ← i + 1
FIN TANT QUE
RETOURNER somme

```

4.2 Boucle POUR (for)

Répète un bloc un nombre déterminé de fois. Le compteur est géré automatiquement.

```

POUR i DE 1 À N FAIRE
AFFICHER "Itération ", i
FIN POUR

```

Exemple — table de multiplication :

```

Algorithme TableMultiplication(N)
POUR i DE 1 À 10 FAIRE
AFFICHER N, " × ", i, " = ", N × i
FIN POUR

graph TD
DEBUT2("Début") --> INIT["i ← 1"]
INIT --> COND3{"i ≤ N ?"}
COND3 -- OUI --> CORPS["Afficher i<br/>i ← i + 1"]
CORPS --> COND3
COND3 -- NON --> FIN2("Fin")
style DEBUT2 fill:#2ecc71,color:#fff

```

```

style COND3 fill:#f39c12,color:#000
style CORPS fill:#3498db,color:#fff
style FIN2 fill:#e74c3c,color:#fff

```

Situation	Boucle recommandée
Nombre d'itérations connu à l'avance	POUR
Parcours d'un tableau de taille N	POUR
Attente d'une condition externe	TANT QUE
Saisie utilisateur jusqu'à valeur sentinelle	TANT QUE

5. Complexité algorithmique

La complexité mesure le nombre d'opérations élémentaires en fonction de la taille n de l'entrée. On s'intéresse au comportement asymptotique (grand n).

```

graph LR
  subgraph "Complexités classiques"
    01["O(1) – Constant<br/>Accès direct"]
    02["O(log n) – Logarithmique<br/>Dichotomie"]
    03["O(n) – Linéaire<br/>Parcours simple"]
    04["O(n log n) – Quasi-linéaire<br/>Tri fusion"]
    05["O(n²) – Quadratique<br/>Boucles imbriquées"]
    06["O(2ⁿ) – Exponentiel<br/>Explosion combinatoire"]
  end
  01 --> 02 --> 03 --> 04 --> 05 --> 06
  style 01 fill:#2ecc71,color:#fff
  style 03 fill:#3498db,color:#fff
  style 05 fill:#e67e22,color:#fff
  style 06 fill:#e74c3c,color:#fff

```

$O(1)$ — Constant : RETOURNER $T[1]$ (1 opération, quel que soit n).

$O(n)$ — Linéaire :

```

Algorithme SommeTableau(T, n)
  somme ← 0
  POUR i DE 1 À n FAIRE
    somme ← somme + T[i]
  FIN POUR
  RETOURNER somme // n additions → O(n)

```

$O(n^2)$ — Quadratique :

```

Algorithme TriBulles(T, n)
  POUR i DE 1 À n-1 FAIRE
    POUR j DE 1 À n-i FAIRE
      SI T[j] > T[j+1] ALORS
        échanger T[j] et T[j+1]
      FIN SI
    FIN POUR
  FIN POUR // n(n-1)/2 comparaisons → O(n²)

```

```

graph TD
  subgraph "Croissance comparée"
    L1["n=10 : O(1)=1, O(n)=10, O(n²)=100"]
    L2["n=20 : O(1)=1, O(n)=20, O(n²)=400"]
    L3["n=50 : O(1)=1, O(n)=50, O(n²)=2500"]
    L4["n=100 : O(1)=1, O(n)=100, O(n²)=10000"]
  end

```

```
style L1 fill:#2ecc71,color:#fff
style L2 fill:#3498db,color:#fff
style L3 fill:#e67e22,color:#fff
style L4 fill:#e74c3c,color:#fff
```

Règle : $O(n)$ est utilisable pour de grandes données, $O(n^2)$ est limite, $O(2^n)$ est inexploitable.

6. Trace d'exécution manuelle

La trace permet de suivre les variables pas à pas pour comprendre et vérifier un algorithme.

Algorithme — Factorielle de N :

```
Algorithme Factorielle(N)
resultat ← 1; i ← 1
TANT QUE i ≤ N FAIRE
    resultat ← resultat × i
    i ← i + 1
FIN TANT QUE
RETOURNER resultat
```

Trace pour $N = 4$: (i, resultat) : (1, 1) → (2, 2) → (3, 6) → (4, 24) → (5, FAUX).

Résultat : Factorielle(4) = 24

QCM d'auto-évaluation

- Quelle est la complexité d'une recherche séquentielle dans un tableau de taille n ?
a) $O(1)$ b) $O(n)$ ✓ c) $O(n^2)$
- Quelle boucle utiliser quand le nombre d'itérations est connu à l'avance ?
a) TANT QUE b) POUR ✓ c) RÉPÉTER
- Quel est le résultat de l'algorithme d'Euclide pour PGCD(12, 8) ?
a) 4 ✓ b) 2 c) 6
- Quelle caractéristique garantit qu'un algorithme ne boucle pas indéfiniment ?
a) Non-ambiguïté b) Finitude ✓ c) Efficacité
- Combien d'opérations un tri bulles effectue-t-il pour $n = 10$?
a) 10 b) Environ 45 ✓ c) 100

Conclusion

L'algorithmique est la science de la résolution de problèmes par des procédures pas à pas. Variables, conditions et boucles sont les briques fondamentales. La complexité ($O(1)$, $O(n)$, $O(n^2)$) permet de comparer les algorithmes. La trace d'exécution est un outil pédagogique essentiel pour déboguer.

Prochaine séance : Structures de données — tableaux, piles LIFO, files FIFO, listes chaînées.

Note pédagogique ENS

Transposition didactique : En SNT Seconde et NSI Première :

- Activité débranchée : Les élèves suivent un algorithme sur papier quadrillé servant de mémoire
- Recette de cuisine : Ingrédients = variables, étapes = instructions, répétitions = boucles
- Tri de cartes : Faire trier des cartes numérotées en suivant le tri bulles ou tri par sélection
- Complexité : Compter le nombre de comparaisons pour différentes tailles de jeu

Lien avec les langues : L'algorithmique peut être comparée à l'analyse grammaticale (une phrase se décompose en sujets/verbes/compléments comme des instructions) et à la rédaction d'un plan de dissertation (structure séquentielle avec des conditions).

Bibliographie

- Cormen, T. et al. (2009). Introduction to Algorithms (3e éd.). MIT Press.
- Dowek, G. et al. (2019). SNT Seconde et NSI Première. Hachette.
- Delannoy, C. (2020). Algorithmique et programmation en Python. Eyrolles.
- Biernat, J. et al. (2020). Algorithmique — cours et exercices corrigés. Dunod.
- Al-Khwârizmî (IXe s.). Al-Jabr wa al-Muqābala.